GUIDELINES FOR GRADE 9

Dynamic Programming

```
dp[i][j]
dp[i-1][j]
dp[i-1][j-2]
```

QUY HOẠCH ĐỘNG

Xem qua bài viết về **đệ quy** trước bài này: Đệ quy & Quay lui.pdf - Google Drive

I. Quy hoạch động *(DP)*

1. Sơ lược về quy hoạch động (QHĐ):

Nhắc lại điểm yếu của đệ quy: trong cây đệ quy ta dễ dàng nhận thấy có không ít bước tính toán bị lặp lại nhiều lần dẫn đến độ phức tạp giải thuật tăng một cách vô ích. “Quy hoạch động” hay “đệ quy có nhớ” sẽ giúp bạn khắc phục điều này.

Định nghĩa: Quy hoạch động được sử dụng khi ta tìm được công thức liên hệ giữa kết quả bài toán có đầu vào cho trước với một (hoặc một số) bài toán con tương tự nhưng có đầu vào nhỏ hơn. Khi ta biết được một số trạng thái bắt đầu của bài toán, ta có thể sử dụng QHĐ để tính ra kết quả cuối cùng.

Nguyên lý hoạt động: là lớp các bài toàn mà quyết định ở bước này phụ thuộc vào các bước đã xử lý trước đó.

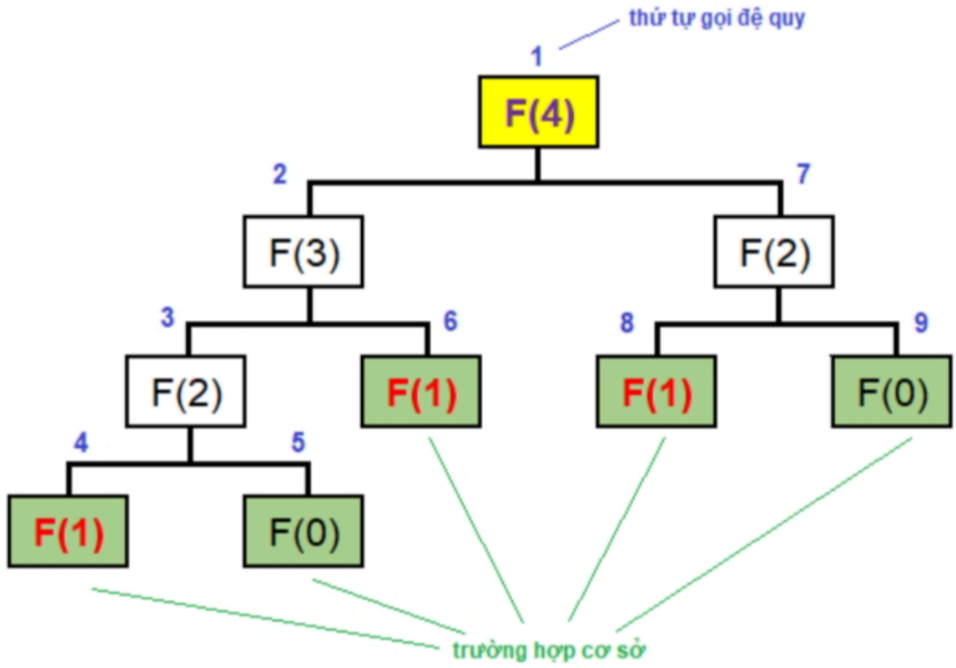
2. Các bước giải bài toán bằng Quy hoạch động:

- Xây dựng hàm mục tiêu ***dp[i]*** là kết quả ở bước thứ ***i***.
- Xác định bài toán cơ sở và công thức truy hồi.
- Lập bảng phương án.
- Kết luận.

3. Ví dụ:

Trở lại một ví dụ quen thuộc về dãy Fibonacci, ta đã biết:

Hãy cùng quan sát và phân tích cây đệ quy với n = 4:



- Cài đặt ngắn gọn, dễ hiểu, dễ phân tích.
- Chỉ với ***n*** = ***4*** mà chương trình phải gọi hàm đệ quy đến ***9*** lần.
- Hàm tính ***F(0)***, ***F(1)*** và ***F(2)*** đều được gọi trên 1 lần. Đặc biệt, hàm tính ***F(1)*** còn được gọi đi gọi lại đến ***3*** lần.

Để hạn chế việc tính lại các bước cũ, ta có thể "ghi nhớ" các kết quả đã được tính qua. Cụ thể, hãy so sánh 2 cách cài đặt sau:

Đệ quy	Quy hoạch động

`int Fibo(int n) {`
`  if (n == 0) return 0;`
`  if (n == 1) return 1;`
`  return Fibo(n-1) + Fibo(n-2); }`
	`cin >> n;`
	`F[0] = 0, F[1] = 1;`
	`for(int i=2; i<=n; i++)`
	`  F[i] = F[i-1] + F[i-2];`
	`cout << F[n];`

Phân tích:

- Hàm mục tiêu: `F[i]` là kết quả của số Fibonacci thứ `i`.
- Bài toán cơ sở: `F[0] = 0`, `F[1] = 1`.
- Công thức truy hồi: `F[i] = F[i-1] + F[i-2]`.
- Bảng phương án:

i	0	1	2	3	4	5	6	7
F[i]	0	1	1	2	3	5	8	13

- Kết luận: `F[n]`.

Nhận xét:

- Ở cách dùng **đệ quy**, ở mỗi bước tính, ta phải gọi lại bài toán con của chính nó; ở mỗi bài toán con, ta lại gọi những bài toán con của bài toán con... Nếu dữ liệu nhập lớn hơn một chút thì chương trình sẽ không xử lý được.

- Ở cách dùng **quy hoạch động**, ta đã tạo mảng để lưu lại các bài toán con đã tính. Như vậy, khi "gọi" lại, ta chỉ cần truy cập vào mảng. Độ phức tạp luôn được đảm bảo ở **O(n)**.

Vận dụng: hãy thử cài đặt bài toán tính giai thừa bằng DP.

II. Dãy con tăng dài nhất *(LIS/LIQ)*

1. Dãy con tăng dài nhất là gì?

Định nghĩa: Dãy con tăng dài nhất (Longest increasing subsequence) là dãy con dài nhất được tạo ra bằng cách xóa một số phần tử *(hoặc không xóa phần tử nào)* của dãy ban đầu mà các vị trí không thay đổi sao cho các phần tử trong dãy con là tăng dần.

Có 2 loại dãy tăng: ngặt và không ngặt. Dãy tăng ngặt là dãy có phần tử **`a[i-1]`** < **`a[i]`** với mọi ($1 \leqslant i \leqslant n$) và tăng không ngặt là dãy có phần tử **`a[i-1]`** $\leqslant$ **`a[i]`** với mọi ($1 \leqslant i \leqslant n$).

2. Áp dụng Quy hoạch động:

Bài toán: Cho dãy số nguyên dương ***a*** gồm ***n*** phần tử *($n \leqslant 1000$)*. Tìm dãy con tăng không ngặt dài nhất (LIQ) của dãy ***a*** *($a[i] \leqslant 100$)*.

Lời giải:

- Dựng hàm mục tiêu ***dp[i]*** là dãy con tăng không ngặt dài nhất tìm được từ ***i*** phần tử đầu tiên của mảng ***a***.
- Trường hợp cơ sở: mỗi phần tử đều là một dãy con không tăng ngặt có độ dài bằng 1.
- Công thức truy hồi: `dp[i] = max(dp[i], dp[j] + 1)`.
  Ta chạy 2 vòng lặp lồng nhau, khi duyệt qua mỗi ***a[i]***, ta kiểm tra nếu ***a[j]*** $\geqslant$ ***a[i]*** (thỏa điều kiện) thì cập nhật ***dp[i]*** *(**dp[j] + 1** là khi ta quyết định ghép phần tử a[i] vào dãy con tăng đang được lưu ở **dp[j]**).*
- Bảng phương án:

A[i]	5	3	7	1	8	6	8
F[i]	1	1	2	1	3	2	4
-->	5	3	57 | 37	1	578 | 378	56 | 36	5788 | 3788

- Kết luận: phần tử lớn nhất của mảng ***dp[]***.

Cài đặt: O(n*n)

```
dp[0] = dp[1] = 1;
for(int i=1; i<=n; i++)
{
    dp[i] = 1;
    for(int j=1; j<i; j++)
        if(a[i] >= a[j])
            dp[i] = max(dp[i], dp[j]+1);
}
for(int i=1; i<=n; i++)
    res = max(res, dp[i]);
cout << res;
```

→ **Tìm các bài tập có cùng tính chất với LIS/LIQ để luyện tập.**

III. Bài toán cái túi *(dp knapsack)*

Bài toán: (ở đây ta chỉ minh họa trường hợp mỗi vật lấy 1 lần).

Trong siêu thị có n đồ vật (n≤1000), đồ vật thứ i có trọng lượng là W[i]≤1000 và giá trị V[i] ≤1000. Một tên trộm đột nhập vào siêu thị, tên trộm mang theo một cái túi có thể mang được tối đa trọng lượng M (M≤1000). Hỏi tên trộm sẽ lấy đi những đồ vật nào để được tổng giá trị lớn nhất.

Giải quyết bài toán trong các trường hợp sau:

- Mỗi vật chỉ được chọn một lần.
- Mỗi vật được chọn nhiều lần (không hạn chế số lần)

InputData: file văn bản **Bag.inp**

- Dòng 1: n, M cách nhau ít nhất một dấu cách
- n dòng tiếp theo: Mỗi dòng gồm 2 số W_i, V_i, là chi phí và giá trị đồ vật thứ i.

OutputData: file văn bản **bag.out**: Ghi giá trị lớn nhất tên trộm có thể lấy

INPUT	OUTPUT
5 15 12 4 2 2 1 1 1 2 4 10	15

Hướng dẫn giải:

Ta nhận thấy rằng, tổng giá trị của cái túi phụ thuộc vào 2 yếu tố: có bao nhiêu vật đang được xét và trọng lượng còn lại mà túi có thể chứa được nên ta có 2 đại

lượng biến thiên. Do đó, hàm mục tiêu phụ thuộc vào 2 đại lượng biến thiên và bảng phương án là bảng 2 chiều.

Gọi ***dp(i, j)*** là tổng giá trị lớn nhất của cái túi khi xét từ vật ***1*** đến vật ***i*** và trọng lượng cái túi chưa vượt quá ***j***. Với giới hạn ***j***, việc chọn tối ưu trong số các vật ***{1, 2, ..., i - 1, i}*** để có giá trị lớn nhất sẽ có 2 khả năng:

- Nếu <u>không chọn</u> vật thứ ***i*** thì ***dp(i, j)*** là giá trị lớn nhất có thể chọn trong số các vật ***{1, 2, ..., i - 1}*** với giới hạn trọng lượng là ***j***, tức là `dp[i][j] = dp[i-1][j]`.
- Nếu <u>chọn</u> vật thứ ***i*** *(phải thỏa **w[i] ≤ j**)* thì ***dp(i, j)*** bằng tổng của giá trị vật thứ ***i*** và <u>giá trị lớn nhất có thể có được</u> bằng cách chọn các vật ***{1, 2, ..., i - 1}*** với giới hạn trọng lượng ***j - w[i]***. Khi đó, ta có: `dp[i][j] = dp[i-1][j-w[i]] + v[i]`.

Chúng ta cần xét xem nếu chọn hay không chọn vật ***i*** thì tốt hơn. Vậy ta xác định được:

- Bài toán cơ sở: `dp[0][j] = 0`.
- CTTH: `dp[i][j] = max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])`.

Bảng phương án:

N/M	*0*	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
2	0	0	**2**	2	2	2	2	2	2	2	2	2	4	4	6	6
3	0	1	2	**3**	3	3	3	3	3	3	3	3	4	5	6	7
4	0	2	3	4	**5**	5	5	5	5	5	5	5	5	6	7	8
5	0	2	3	4	10	12	13	14	**15**	15	15	15	15	15	15	15

Vậy chúng ta có thể chọn vật 2, 3, 4, 5.

Cài đặt: O(n*m)

```
memset(dp , 0 , sizeof(dp));
for(int i = 1 ; i <= n ; i++) cin >> w[i] >> v[i];
for(int i = 1 ; i <= n ; i++)
{
    for(int j = 1 ; j <= m ; j++)
    {
        dp[i][j] = dp[i-1][j];
        if(j >= w[i])
            dp[i][j] = max(dp[i][j] , dp[i-1][j - w[i]] + v[i]);
    }
}
cout << dp[n][m];
return 0;
```

Nguồn: tham khảo phần *Phương pháp Quy hoạch động* trong sách *Giải thuật và Lập trình* (thầy Lê Minh Hoàng).

Vận dụng: Vẫn bài toán trên nhưng trường hợp mỗi vật được lấy nhiều lần. Tìm các bài tập có tính chất tương tự để tìm hiểu thêm.

IV. Dặn dò

✏️ Quy hoạch động có vô số dạng khác nhau, có thể xem QHĐ là một trong những "ác ma" của Lập trình thi đấu. Tuy nhiên, bài viết này chủ yếu cho bạn hiểu về ý tưởng của quy hoạch động, giúp bạn cài được những bài quy hoạch động **cơ bản** có thể ra trong TS10.

✏️ Bởi vì thời gian cho đến ngày thi không còn nhiều, không đảm bảo đủ để các bạn ôn luyện DP (đối với người mới học) nên bọn mình quyết định đăng tải một số bài tập quy hoạch động *(lưu ý là đăng ở dạng bài tập, **không** phải contest)* dành cho các bạn đủ khả năng làm và muốn làm. Và tất nhiên solutions sẽ có sau kỳ thi TS10 để dành thời gian cho việc hỗ trợ các bạn 2k9 ôn thi.

✏️ Đón chờ các bài luyện tập sắp tới tại **GDOJ**.

✏️ Đón chờ các contest pre TS10 sắp tới tại **GDOJ**.

✏️ Tham gia **đoạn chat cộng đồng** để giao lưu cùng các bạn đồng môn và các anh chị học sinh chuyên Tin trên TPHCM.

✏️ Nguồn tham khảo của bài viết: **VNOI**

✏️ Thông tin liên hệ:
- Page: **Căng TIN 2326**
- Group: **Guidelines for Grade 9**
- Gmail: **cangtin2326@gmail.com**

--- Hết ---